羊城杯 2026 · PWN · cacheline

缓存行中的跨对象生命周期利用 —— 三层交叉认证对象图的打穿过程

题目	cacheline（MEDIUM，初始 500 分，当前 454 分，106 队解出）
附件	nginx 1.31.6 / cacheline.so（nginx 动态模块）/ cachebroker / cachevault / ld-musl / layout_shuffle.so
目标	从 vault（uid 1002）取回 /home/vault/flag，flag 在 vault 内被 dup2 到 fd 7
本地	WSL Ubuntu-24 + Docker（alpine 3.22）完整复现；exp 在本地打通到第 3 层握手
远程	1.14.76.59:21426（代理到容器 9999）；实例 expireTime=1792834200 = 2026-10-24 09:30 UTC，本次分析时已过期，后端直接关闭连接

说明：本报告给出三层的完整逆向结论（协议、对象布局、认证公式、密钥泄漏点）与层 0/1 的可用利用代码。层 2 的最终取 flag 需要控制流劫持 + ROP，报告给出精确的利用原语与缺口。

1. 目标结构与信任模型

三个进程串成一条“逐层放行”的管道，只有上一层完成认证，连接才会被移交给下一层：

```
client --HTTP/1.1(keepalive)--> nginx worker (加载 cacheline.so, 非 root)
                                    |  connect(/run/cacheline-broker.sock, 0666)
                                    v
                               cachebroker (setuid 1001)
                                    |  connect(/run/cacheline-vault.sock, 0666)
                                    v
                               cachevault  (setuid 1002, open(/home/vault/flag) -> dup2(fd,7))
```

三层内部都是同一套设计：程序负责分配对象、写入随机种子与“按地址绑定”的 cookie，而“导入”（import）命令允许攻击者把整块对象内容写回去。认证通过后，层内会执行一个间接调用（call rax），把当前 socket 交给下一层。

因此本题的核心不是栈溢出，而是：用泄漏出来的密钥重新计算 MAC 与 cookie，伪造出满足交叉认证的对象图，再把这个层内的“合法间接调用”指向下一层的入口。

2. 本地靶场复现

远程只给了一个 nc 端口，没有给 nginx.conf。第一步是把三层跑起来，否则无法动态验证任何假设。

2.1 关键发现：模块必须由指令启用

cacheline.so 里有一个 0 参数指令 cacheline（命令表在 .data+0x6100，名字串 .rodata+0x40de）。它的 set 回调（0x1ac8）会取 ngx_http_core_module 的 loc_conf，把 clcf->handler(+0x68) 设为模块的 content handler 0x2199。只 load_module 而不在 location 里写 cacheline; 时，请求仍然走静态文件逻辑（返回 404）。

```
load_module /opt/cacheline/cacheline.so;
worker_processes 1;                  # 模块状态在 worker 的 .so 静态区，必须单 worker
events { worker_connections 128; }
http {
    server { listen 9999;
        location / { cacheline; }      # 关键：启用模块
    }
}
```

2.2 容器与用户

镜像基于 alpine:3.22（与附件一致的 musl 1.2.6 / mallocng），安装 pcre2/openssl/zlib，创建 uid 1001(broker)、1002(vault)、101(nginx)，把 flag 写进 /home/vault/flag（400 vault）。entry.sh 依次拉起 vault、broker、nginx。Dockerfile/entry.sh/nginx.conf 与 exp.py 一起提交在题目目录。

2.3 协议入口

handler 只接受 GET（r->method & GET），从 query 里取两个参数：op 与 data（data 是十六进制串，模块用 ngx_hextoi 每 2 字符解 1 字节）。状态机保存在 .so 的静态变量 level(.so+0x6600) 里，所以所有请求必须复用同一条 HTTP keepalive 连接、且落在同一个 worker。

3. 防护机制

项	结论
ELF	三个程序均为 PIE；Full RELRO（BIND_NOW + GNU_RELRO）；GNU_STACK RW（NX）
libc	musl（Alpine, GCC 15.2 / musl-1.2.6），分配器是 mallocng：按 size-class 分组，槽位用位图管理，槽内不写元数据
提权	broker/vault 启动后 setgid+setuid 到 1001/1002；socket 0666 供 nginx worker 连接
seccomp	broker/vault 各自装 BPF 过滤器：校验 arch=x86_64，nr>=0x40000000 直接 kill，execve(0x3b)/execveat(0x142)（vault 还含 openat 0x101）返回 EPERM，其余放行
其它	alarm(7200)；vault 把 flag fd dup2 到 7 后关闭原 fd；broker 在连接关闭时由 pool cleanup 里 _exit(0)

4. 通用构件：哈希与“按地址绑定”的凭证

三层用的是同一个哈希（broker 0x2260 / 模块 0x115b / vault 0x1410，逐指令相同）：splitmix64 finalizer 做压缩，数据块逐个混入，key2 每轮累加常数，轮转位数每轮 +0xb。它同时被用来做“完整性标签”（MAC）和“密钥派生”。

```
def F(x):                                  # splitmix64 finalizer
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9 & M64
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb & M64
    return x ^ (x >> 31)

def H(k1, k2, data, n):                    # data: n 个 u64
    h = F(k1 ^ n ^ 0x9e3779b97f4a7c15)
    kk = k2
    for i in range(n):
        t = F(kk + data[i]);  kk += 0x517cc1b727220a95
        h = ((k1 ^ (i * 0x6a09e667f3bcc909)) + rol(t ^ h, 0x11 + 0xb*i)) & M64
    return F(k2 ^ h ^ 0xa54ff53a5f1d36f1)
```

每个对象创建时会写入两类“凭证”，是整个题目可利用性的根源：

字段	含义	为什么可被利用
cookie = KEY ^ obj_addr	把对象与它的堆地址绑定	只要能把泄漏到的 cookie 原样填回同一地址的对象，就能伪造凭证
mac = H(k1, k2, obj[0..0x40))	覆盖对象自身字段的完整性标签	layered 泄漏把 k1/k2 交出来之后，MAC 可以离线计算
key = H(seed, 常量, {ctx, code_ptr})	每层/每连接的随机密钥	每层都提供了一个“已知数据 ^ 密钥”的泄漏点

5. 第 0 层：nginx 模块（cacheline.so）

5.1 状态机

level	op（query 参数）	data 长度	动作 / 应答
0	create	0x300	分配 0x180 字节 CLTACKT4 对象 → created

level	op（query 参数）	data 长度	动作 / 应答
1	resize	0x300	再分配 0x180；只有 malloc 复用同一地址且 [0x38]=="CLTACKT4"、[0x40]==0x491 才继续：创建 beacon(0x128) 与之后 ledger，回 staged；否则 queued
2	ledgeropen	-	校验 beacon，创建 CLLEDGR4(0xf8) → opened（注意 op 是 10 字节的 "ledgeropen"，不是 ledger）
3	beacon	-	回 "n= a=<...> e=<...> k=<...>"（beacon 字段泄漏）
3	ledger	-	回 "n=o=t="（ledger 字段泄漏）
3	route	0x350	校验 CLLROUT4(0x1a8) → routed
4	seal	0x1b0	校验 CLLSEAL4(0xd8) → sealed
5	grant	0x430	校验 CLGRANT4(0x218) → granted
6	voucher	0x210	校验 CVOUCHR4(0x108) → vouched
7	commit	-	重校验 + 异或条件，注册连接清理回调 → activated（level=8）
8	broker	任意	hex 解码后写 broker socket（模块自动加 2 字节大端长度前缀），并把回复作为 HTTP body 返回

5.2 “resize”为什么能成功：mallocng 的槽位复用

每个请求结束时，模块用 pool cleanup 把 alloc 出来的对象 free 掉；下一个请求同样大小分配时，mallocng 会从该 size-class 分组的空闲位图里取槽位。不保证立刻返回“刚释放”的那个槽位，所以 create 之后的 resize 常常先返回 queued，多试几次（实测 3~5 次）就会拿到同一地址。本层第一次“跨对象生命周期”的利用就体现在这里：模块把静态指针指向的旧对象，已经被下一次请求的数据覆盖，而模块仍然把它当 tack 用。

5.3 密钥泄漏

beacon 创建时（0x24bf 起）写入的 4 个字段全部被打印出来，其中三个与密钥相关；注意 0x2199 在代码里是 RIP 相对 lea，取到的是绝对地址 base+0x2199：

```
a = rol(seed, 0x11) ^ (base + 0x2199)                  -> base = (a ^ rol(seed,0x11)) - 0x2199
e = H(base+0x2199, seed, [seed, base+0x2199], 2) ^ K2    -> K2 = e ^ H(...)
k = H(base+0x2199, seed, [K2, seed], 2)          ^ K1    -> K1 = k ^ H(...)
ledger: o = H(K1,K2,[130, seed],2) ^ conn                -> conn(nginx connection 指针) = o ^ H(...)
```

拿到 K1/K2/conn/base 之后，本层所有 MAC 都能离线算。

5.4 伪造对象与验证条件

对象/长度	关键字段与验证	MAC
route 0x1a8	[0x38]="CLLROUT4", [0x70]=1, [0x50]=beacon.seed, [0x58]=ledger[0x30], [0x68]≠0; [0x48]=H(K1,K2,{[0x40],seed,ledger[0x30]},3) ^ conn	[0x78]=H(K1,K2,{tag,[0x40],[0x48],[0x50],[0x58],[0x60],[0x68],1},8)
seal 0xd8	[0x28]="CLLSEAL4", [0x30]=route[0x68], [0x38]=route[0x40], [0x40]=seed, [0x48]=ledger[0x30], [0x58]=1	[0x60]=H(K1^0x243f...,K2,{CLLSEAL4,[0x30..0x50],1},7)
grant 0x218	[0x38]="CLGRANT4", [0x50]=seed, [0x58]=ledger[0x30], [0x60]=route[0x40], [0x68]=seal[0x30], [0x88]=1	[0x80]=H(K1^K2,K1+K2,{0x2199abs,conn,seed,130,route[0x40],seal[0x30],route[0x60],seal[0x50]},8); [0x90]=H(K1^0x1319...,K2,{11 个 qword},11)
voucher 0x108	[0x28]="CVOUCHR4", [0x30]=grant[0x78], [0x38]=grant[0x40], [0x40]=route[0x40], [0x48]=seal[0x30], [0x50]=seed, [0x58]=130, [0x68]=1	[0x70]=H(K1^0x3c6e...,K2,{9 个 qword},9)
commit 条件	route[0x60] ^ seal[0x50] ^ grant[0x70] ^ voucher[0x60] == 1	route[0x60] 可自由选择，用它凑出 1

commit 通过后：call [0x5e00+idx*8]，idx 由上式的异或结果决定；idx==1 时调用 0x1aef，它给当前 connection 注册 cleanup（连接关闭时 close(broker_fd) 并 _exit(0)），返回 0，回 activated。

5.5 踩坑记录

① op 字符串是 ledgeropen（10 字节），不是 ledger —— .rodata 里 "ledger" 和 "open" 紧邻，编译后合并成 10 字节常量；② 0x2199 是绝对地址，用常数 0x2199 算 e/k 会解出错误密钥；③ reverse 时容易把 resize 的成功条件看成“tag/size 正确”就够了，实际还需要指针复用。

6. 第 1 层：cachebroker

6.1 对象表（.bss）

主表	对象	链接表	import 标记	配对
0x7420	CLPROBE4 0x90	0x73e0	0x73a0	N/L/X/A、P
0x7360	CLAPERT4 0xc0	0x7320	0x72e0	n/l/x/a、p
0x71e0	CLSHARD4 0xf0	0x71a0	0x7160	S/T/W/F、U
0x7120	CLSTAMP4 0x130	0x70e0	0x70a0	s/t/w/f、u
0x72c0	CPERMIT4 0x160	0x72a0	0x7280	G/J/H/K、I
0x7260	MANDATE4 0xb0	0x7240	0x7220	M/R/O/Y、Q

命令是 2 字节长度前缀 + 负载；3 字节命令（N/n/L/l/B/V/S/s/T/t/H/O/D/E）带两个下标，2 字节命令（P/X/p/x/G/I/J/M/Q/R/U/W/u/w）带一个下标，import 命令（A/a/F/f/K/Y）用消息总长度区分（0x92/0xc2/0xf2/0x132/0x162/0xb2），写回对象的字节数正好等于对象大小。

6.2 令牌泄漏

'V' 要求两边的 import 标记非空，然后对（探针，孔径）做完整交叉认证（tag、[0x30]==1、cookie=K1^ptr、互相指向对方的 seed 与 cookie、双方 MAC），通过后按类型返回 8 字节：

```
type 0 -> *(broker+0x7008) = base + 0x4000      # 结束函数 lea 出的 PIE 地址，泄漏基址
type 1 -> K1                                     # cookie 与多种 MAC 的 key1
type 2 -> K2                                     # 另一种 MAC 的 key2
```

6.3 关键技巧：import 会破坏 cookie，所以先泄漏再写回

'A'/'a' 这类命令把整块对象覆盖成攻击者的数据，会把创建时写入的 cookie([0x18]=K1^ptr) 一起冲掉。但由于 import 前后对象地址不变，正确顺序是：先 'P'/'p' 把 0x40 字节（含 cookie）读出来，再把同一个 cookie 原样写进 import 数据，最后用 'B' 让程序重算交叉字段与 MAC。

6.4 K0 与最终对象图

```
K0 = H(K1, K2, [base+0x4000, 0x4f524b4559313233], 2)      # .data 里的 {proxy 指针, "321YKRO"}

shard[0x30] = H(K0, seed, [i], 1)  then 21 轮
              H(K0^seed, K1, [h, payload_qword, byte_offset], 3)   # 第三元素是字节偏移 0,8,...,0xa0
stamp[0x30] = 同一个值
permit[0x28] = 4 轮 H(K0^i, K1, [h, shard[0x30], shard[0x10], stamp[0x10], shard_ptr, stamp_ptr], 6)
mandate[0x30] = H(K2^K1, K0, [h4, permit_ptr, mandate_ptr, permit[8], mandate[8]], 5)
```

'C' 的最终调用是 rax = permit[8] ^ mandate_ptr ^ K2 ^ ror(mandate[0x28],0x13)；把 mandate[0x28] 设成 rol(目标 ^ K2 ^ mandate_ptr ^ permit[8], 0x13)，rax 就能指向任意地址。这里指向 base+0x4000 —— broker 的 proxy 函数：connect(/run/cacheline-vault.sock)，写 16 字节 "CLV4/VAULT\n"+5 字节固定尾巴，然后 poll 双向转发。

实测（本地）：

```
[broker] base=0x5fb10fe17000 K1=0x0b5e5eff4bf299f8 K2=0x86e0f59823fbbb92
[broker] C -> 200 b'CLR4p\x00\x00\x00\t\x00\x00\x00\x00\x00\x00\x00CV4/READY'
```

这一条 11 字节 'C' 消息同时也满足了模块层 n==0xb && ■■■ 4 ■■=="CLR4" 的检查，模块把 level 提升到 9，之后 op=broker 的数据不再加长度前缀，直接透传给 vault。

7. 第 2 层：cachevault

7.1 帧格式与上下文

```
请求: "CLV4" | op | a | b | 0 | length(le32, <=0x600) | seq(le32, 必须递增)
      + payload(length 字节)
应答: "CLR4" | code | a | b | 0 | (len<<32)|0x00ffffffffffffff | payload
```

握手由 broker 完成：vault 读 16 字节，与 .rodata+0x5130 的 "CLV4/VAULT\n\x91\x4e\x2b\xc7\x5d" 比较，然后 open(/home/vault/flag) 并把 fd dup2 到 7，calloc(0xf8) 出上下文对象（全局 0x76a0）：

```
ctx[0]    = getrandom(8)                    # seed
ctx[8]    = ctx                             # 自指针
ctx[0x10] = base + 0x17f0                   # .data.rel.ro 里的函数指针
ctx[0xd8] = H(seed, 0x6a09e667f3bcc909, {ctx[8], ctx[0x10]}, 2)
ctx[0xe0] = H(seed, 0xbb67ae8584caa73b, {ctx[8], ctx[0x10]}, 2)
ctx[0xe8] = H(seed, 0x3c6ef372fe94f82b, {ctx[8], ctx[0x10]}, 2)
ctx[0xf0] = H(seed, 0xa54ff53a5f1d36f1, {ctx[8], ctx[0x10]}, 2)
```

7.2 opcode 表与对象表

op	作用（与 broker/模块同构）
0x01	执行“plan”：payload 为 0x10 字节记录的序列，每条记录 word2 = (0x6d*i & 0xffff) ^ 0x4b27，记录类型 A/S/g/0x8b 各有语义
0x02	知识证明：len==8，比较 payload 与 H(ctx[0], ctx[8], {ctx[0x10], 0x243f6a8885a308d3}, 2)
0x03/0x04/0x06/0x08	VMTOKEN(0xa0)@0x7660 与 VMRECPT(0xd0)@0x75a0 的创建/链接/释放
0x05/0x07	泄漏 0x40 字节 / import 0xa0 字节（标记表 0x75e0/0x7520）
0x0d	VMTOKEN 与 VMRECPT 配对：写 cookie 与互相指向的字段，重算 MAC 0x9a
0x10..0x1a	CTICKET(0x150)@0x74a0、VALCRED(0xb0)@0x7420 的创建/链接/泄漏/import
0x20..0x24	CLVSEAL(0x98)@0x7120 的创建/链接/泄漏/import
0x2c	把 0x7360[a] 与 0x72a0[b] 绑定：obj2[0x28]=rol(0x17f0abs ^ ctx[0xe0] ^ obj2 ^ obj1[8],0x15)
0x30..0x3a	CLCHUNK(0x140)@0x71e0 的创建/链接/import/泄漏/释放/配对
0x40	最终 commit：校验 4 组对象 + 链式摘要 → call rax

0x10 及以上的 opcode 会被 [0x7084] 这个状态门控（为 0 时统一回 code 0xe2），也就是说必须按 vault 自己的顺序把前面的层做完。

7.3 最终 commit（0x28e7）与它的问题

payload 只有 0xc 字节：前 4 字节是 4 个下标（依次索引 0x7460/0x73a0/0x72e0/0x7220 的标记表与 0x74a0/0x73e0/0x7320/0x7260 的对象表），后 8 字节是 4 组 (d,e) 下标，指向 0x71a0/0x70e0。依次做：许可/授权交叉校验 → 4 组配对校验 → 链式摘要（H(ctx[0xe8]^i, ...) 交替）→ 把 4 个对象各 0x100 字节复制到第 3 个对象 +0x40（潜在越界写）→ 计算并调用：

```
rax = [obj3+8] ^ obj4 ^ ctx[0xe0] ^ ror([obj4+0x28], 0x15)
if (rax >> 0x2f) not in {0, 0x1ffff}: _exit(1)
call rax            ; rdi = obj3
```

若按 0x2c 的绑定来填，rax 恰好化简为 base+0x17f0 —— 一个空操作函数（与 broker 的 0x2580 同款设计）。换句话说，合法路径只会“放行”，不会给 flag。

7.4 取 flag 的路径：栈迁移 gadget + 相对地址技巧

vault 里只有一处 read 调用（0x1a38，fd 固定为 0），没有任何 read(7)；所有输出都走 0x18d0（"CLR4"+code 的应答）。但二进制末尾保留了 4 个刻意的 gadget（.text 0x44e0 起）：

```
0x44e0: lea rsp,[rdi+0x40] ; ret      <-- 栈迁移: rsp = rdi+0x40
0x44f0: pop rdi ; ret
0x4500: pop rsi ; ret
0x4510: pop rdx ; ret
```

而 commit 在 call rax 时 rdi 正好等于第 3 个对象指针，且它前面刚把 4 个对象各 0x100 字节复制到 rdi+0x40 —— 也就是说 rdi+0x40 起的 0x400 字节完全由我们的 import 数据构成，正好是 ROP 链的落地位置。所以目标是：rax = base+0x44e0，链为 pop rdi(7) / pop rsi(buf) / pop rdx(len) / read@plt /

write@plt(1,...)。

不需要泄漏 PIE 基址的技巧：op 0x2c 的绑定把 obj4[0x28] = rol((base+0x17f0) ^ ctx[0xe0] ^ obj4 ^ obj3[8], 0x15)，于是 commit 算出的 rax 恒等于 base+0x17f0。若能在绑定之后把 obj3[8] 改成 ■■ ^ (0x17f0 ^ 0x44e0) = ■■ ^ 0x5310，rax 就会变成 base+0x44e0 ——
这是个纯常量偏移，不需要知道基址。剩下的难点是：commit 会对 obj3/obj4 做交叉认证，必须找到一种只改动 obj3[8]、又能让 MAC 仍然成立的写入方式（例如利用 commit 自身 4x0x100 的越界写去覆盖相邻对象，或先 import 再让程序自己重算 MAC）。

另外，op 0x10 及以上的命令被 [0x7084] 这个门控变量挡着（回 code 0xe2），唯一把它置 1 的地方是 op 0x02 成功之后：op 0x02 要求 8 字节 payload 等于 H(ctx[0], ctx[8], {ctx[0x10], 0x243f6a8885a308d3}, 2)，即用 seed 与 ctx 地址做 MAC。因此完整利用还需要先解出 op 0x01(plan VM)/0x03..0x0d(VMTOKEN/VMRECPT)
这条支线，从对象字段回显中把 ctx 的 seed/地址带出来，再打开门控。

8. 漏洞本质与可复用经验

题目把“对象生命周期”与“缓存行/分配器行为”绑定在一起，三层都在同一个模式上做文章：

① 按地址绑定的 cookie：cookie=KEY^addr 看似能防伪造，但只要 import
会覆盖对象、而对象地址不变，就可以把泄漏到的 cookie 原样回填；② import
命令没有类型检查，只检查链接表非空与标记表为空，于是“链接表里的指针”就是最好的写原语；③ malloc
复用：模块层用 free→realloc 让静态指针指向攻击者数据（resize 的指针相等检查就是提示）；④ 层间密钥以 MAC 形式泄漏：每层都有“已知数据 ^ 密钥”的字段被打印出来，把不可伪造的认证链变成可离线重算的方程组。

复现时最花时间的三件事：确认模块需要 cacheline; 指令才能启用；确认 0x2199
是绝对地址；确认分片链式摘要的第三个元素是字节偏移而不是下标。

9. 交付物与复现步骤

```
D:\桌面\羊城杯\pwn\cacheline\
  exp.py                  # 层0+层1 完整利用 + 层2 客户端框架
  cacheline_解题思路.pdf   # 本文档
  work\Dockerfile         # 本地靶场（alpine 3.22 + pcre2/openssl/zlib + 三个用户）
  work\entry.sh           # 依次启动 vault/broker/nginx
  work\nginx.conf         # load_module + location { cacheline; }
  analysis\*.asm          # 三个二进制的反汇编

# 本地
docker build -t cacheline-env work/
docker run -d --name cl -p 19999:9999 cacheline-env
python3 exp.py 127.0.0.1 19999

# 远程（实例有效时）
python3 exp.py 1.14.76.59 21426
```

9.1 远程状态

题目描述里的 expireTime=1792834200 即 2026-10-24 09:30 UTC（北京时间 17:30）。本次分析进行到 10:47 UTC 时，连接 1.14.76.59:21426 可以完成 TCP 握手，但约 4.7 秒后对端直接 close，不返回任何字节 ——
后端容器已被回收。因此报告中的远程验证无法完成，所有结论均在本地容器中动态验证过。

附录 A：本地验证记录（摘录）

```
# 层 0
[so] resize reuse after 4 tries
[so] module_base+0x2199=0x74ed500fc199 K1=0x337e251954ba051d K2=0x977878e7cca741c1 conn=0x74ed5007a420
[so] module gate opened (level=8)
# 层 1
[broker] base=0x5fb10fe17000 K1=0x0b5e5eff4bf299f8 K2=0x86e0f59823fbbb92 K0=0x0da85a0fd5efdb48
[broker] permit=0x7ca41b0458a0 mandate=0x5fb10fe1ed70
[broker] C -> 200 b'CLR4p\x00\x00\x00\t\x00\x00\x00\x00\x00\x00\x00CV4/READY'
[+] pivot to vault, greeting = b'CLR4p...CV4/READY'
# 层 2（握手后第一次探测）
[vault] op=0x15 -> b'CLR4\xe2\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00'   # 0xe2 = 状态门未开
```

附录 B：层 0 校验用到的全部 key 组合

用途	k1	k2	覆盖字节
CLTACKT4 / beacon / ledger 内部 MAC(0x121b/0x126b)	K1(0x6608)	K2(0x6610)	5 或 6 个 qword
CLLROUT4 MAC	K1	K2	8 个 qword
CLLSEAL4 MAC	K1 ^ 0x243f6a8885a308d3	K2	7 个 qword
CLGRANT4 MAC#2	K1 ^ 0x13198a2e03707344	K2	11 个 qword
CLGRANT4 MAC#1	K1 ^ K2	K1 + K2	8 个 qword
CVOUCHR4 MAC	K1 ^ 0x3c6ef372fe94f82b	K2	9 个 qword
ledger 字段 / beacon e,k	K1	K2 / 见正文	2 或 4 个 qword

附录 C：层 1 的全部 MAC

对象	MAC 位置	k1	k2	qword 数
CLPROBE4	+0x38	K3(0x7098)	K1	7
CLAPERT4	+0x38	K3 ^ 0x243f6a8885a308d3	K1	7
CPERMIT4	+0x38	K2	K1	7
MANDATE4	+0x40	K2 ^ 0x13198a2e03707344	K1	8
CLSHARD4	+0x40	K1	K0	8
CLSTAMP4	+0x40	K1 ^ 0x3c6ef372fe94f82b	K0	8

附录 D：参考地址（相对文件偏移）

```
cachebroker : 0x2000 main / 0x26b0 session / 0x2260 hash / 0x2370..0x2520 六个对象 MAC
              0x25e0 分片链式摘要 / 0x4000 proxy->vault / 0x7008 .data = base+0x4000
cacheline.so: 0x2199 content handler / 0x115b hash / 0x18d0 组包发送 / 0x1aef 注册 cleanup
              0x1cb0 broker 中继 / 0x1ac8 "cacheline" 指令 set 回调 / 0x1f02 worker init(seccomp+key)
cachevault  : 0x1ab0 session / 0x1410 hash / 0x18d0 组包发送 / 0x17f0 空操作函数
              0x28e7 op 0x40 commit(含 call rax) / 0x25bd op 0x01 plan / 0x2c15 op 0x2c 绑定
              0x76a0 ctx 对象 / ctx[0xd8..0xf0] 四个派生密钥
```